

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

INGENIERÍA EN COMPUTACIÓN



NIM — MINIMAX

Juego de Nim multi-pila con algoritmo Minimax

PASQUEL JOHANN — TORRES JORGE — GUACHAMÍN
DAVID — CEDENO MATHEW

Inteligencia Artificial (ICCD563)

Quito, Ecuador
23 de enero de 2026

Índice

1. Introducción	2
1.1. Objetivos del Proyecto	2
2. El Juego Nim Multi-Pila	2
2.1. Reglas del Juego	2
2.2. Representación del Estado	2
2.3. Estrategia matemática: Nim-sum	3
2.4. Cálculo del Movimiento óptimo	4
2.5. Ejemplo de Cálculo	4
2.6. Tabla de posiciones	5
3. Algoritmo Minimax	5
3.1. Fundamentos	5
3.2. Implementación con Poda Alpha-Beta	6
3.3. Complejidad	6
4. Implementación del Agente	6
4.1. Arquitectura del Sistema	6
4.2. Niveles de Dificultad	7
4.3. Proceso de Decisión	7
5. Resultados Experimentales	7
5.1. Metodología	7
5.2. Análisis de Resultados	8
5.3. Observación Importante	8
6. Conclusiones	8
6.1. Logros del Proyecto	8
6.2. Observaciones Clave	8
A. Instrucciones de Uso	9
A.1. Instalación	9
A.2. Ejecución	9
B. Estructura del Repositorio	9

1. Introducción

El presente documento describe la Implementación de un agente inteligente para el juego de Nim con multiples pilas utilizando el algoritmo Minimax. El proyecto demuestra la aplicacion de la teoria de juegos combinatorios y tecnicas de inteligencia artificial en un juego de estrategia clasico.

1.1. Objetivos del Proyecto

Los objetivos principales del proyecto son:

1. Implementar el juego de Nim con tres pilas y reglas de seleccion de fila.
2. Desarrollar un agente inteligente basado en el algoritmo Minimax.
3. Aplicar la estrategia matemática optima basada en Nim-sum.
4. Proporcionar una interfaz grafica interactiva con visualizacion 3D.
5. Analizar el rendimiento del agente mediante simulaciones.

2. El Juego Nim Multi-Pila

2.1. Reglas del Juego

definición 2.1 (Juego de Nim Multi-Pila). El juego de Nim con multiples pilas se define como sigue:

- **Estado inicial:** Tres pilas con 3, 5 y 7 palitos respectivamente (total: 15).
- **Jugadores:** Dos jugadores que alternan turnos.
- **Movimientos validos:** En cada turno, el jugador debe:
 1. Seleccionar una pila no vacia.
 2. Remover de 1 a 3 palitos de esa pila.
- **Condición de victoria:** El jugador que remueve el **ultimo palito gana**.

2.2. Representación del Estado

El estado del juego se representa mediante una tupla de tres enteros:

$$S = (n_1, n_2, n_3) \in \mathbb{Z}_{\geq 0}^3 \tag{1}$$

donde n_i representa el número de palitos en la pila i .

El estado inicial es $S_0 = (3, 5, 7)$ y el estado terminal es $S_f = (0, 0, 0)$.

2.3. Estrategia matemática: Nim-sum

La solución matemática del juego de Nim fue descubierta por Charles L. Bouton en 1901. La estrategia se basa en el concepto de *Nim-sum*, que es la operación XOR (o exclusiva) aplicada a los tamaños de las pilas.

definición 2.2 (Nim-sum). Para un estado $S = (n_1, n_2, n_3)$, el Nim-sum se define como:

$$\text{Nim-sum}(S) = n_1 \oplus n_2 \oplus n_3 \quad (2)$$

donde $\oplus$ denota la operación XOR bit a bit.

Teorema 2.1 (Estrategia Óptima de Nim - Bouton, 1901). En el juego de Nim:

1. Una posición es **perdedora** (posición P) si y solo si $\text{Nim-sum}(S) = 0$.
2. Una posición es **ganadora** (posición N) si y solo si $\text{Nim-sum}(S) \neq 0$.
3. Desde una posición ganadora, siempre existe un movimiento que lleva a una posición perdedora.

Demostración. La demostración utiliza inducción sobre el número total de palitos:

Caso base: El estado $(0, 0, 0)$ tiene $\text{Nim-sum} = 0$ y es una posición perdedora (el jugador anterior gana).

Paso inductivo:

- Si $\text{Nim-sum}(S) = 0$, cualquier movimiento m produce un estado S' con $\text{Nim-sum}(S') \neq 0$. Esto se debe a que cambiar el valor de una sola pila necesariamente altera el XOR total.
- Si $\text{Nim-sum}(S) \neq 0$, sea k el bit más significativo de $\text{Nim-sum}(S)$. Existe al menos una pila n_i que tiene el bit k activo. Removiendo palitos de esta pila para que $n'_i = n_i \oplus \text{Nim-sum}(S)$, se obtiene $\text{Nim-sum}(S') = 0$.

□

2.4. Cálculo del Movimiento óptimo

Algorithm 1 Cálculo del Movimiento óptimo en Nim

```

1: function MOVIMIENTOÓPTIMO( $n_1, n_2, n_3$ )
2:    $nimSum \leftarrow n_1 \oplus n_2 \oplus n_3$ 
3:   if  $nimSum = 0$  then
4:     return null ▷ posición perdedora
5:   end if
6:   for  $i \leftarrow 1$  to 3 do
7:      $objetivo \leftarrow n_i \oplus nimSum$ 
8:     if  $objetivo < n_i$  then
9:        $remover \leftarrow n_i - objetivo$ 
10:      if  $1 \leq remover \leq 3$  then
11:        return ( $i, remover$ )
12:      end if
13:    end if
14:  end for
15:  return null
16: end function

```

2.5. Ejemplo de Cálculo

Para el estado inicial (3, 5, 7):

$$3 = 011_2 \quad (3)$$

$$5 = 101_2 \quad (4)$$

$$7 = 111_2 \quad (5)$$

$$\text{Nim-sum} = 001_2 = 1 \quad (6)$$

Como Nim-sum $\neq 0$, es una posición ganadora. El movimiento óptimo es remover 1 palito de la fila 1, dejando el estado (2, 5, 7):

$$2 = 010_2 \quad (7)$$

$$5 = 101_2 \quad (8)$$

$$7 = 111_2 \quad (9)$$

$$\text{Nim-sum} = 000_2 = 0 \quad (10)$$

Ahora el oponente está en posición perdedora.

2.6. Tabla de posiciones

Cuadro 1: Ejemplos de posiciones ganadoras y perdedoras

Estado	Nim-sum	Tipo	Movimiento óptimo
(3, 5, 7)	1	Ganadora	Fila 1: quitar 1
(2, 5, 7)	0	Perdedora	—
(1, 1, 0)	0	Perdedora	—
(2, 2, 0)	0	Perdedora	—
(1, 2, 3)	0	Perdedora	—
(2, 3, 4)	5	Ganadora	Fila 3: quitar 1

3. Algoritmo Minimax

3.1. Fundamentos

Aunque la estrategia de Nim-sum proporciona una solución analítica óptima, se implementó también el algoritmo Minimax para demostrar su aplicabilidad en juegos de suma cero.

definición 3.1 (Algoritmo Minimax). El algoritmo Minimax es una técnica de búsqueda adversarial que:

- Construye un árbol de juego con todos los estados posibles.
- Asigna valores a estados terminales (+1 victoria, -1 derrota).
- Propaga valores hacia arriba: MAX elige máximo, MIN elige mínimo.

3.2. Implementación con Poda Alpha-Beta

Algorithm 2 Minimax con Poda Alpha-Beta

```

1: function MINIMAXAB(estado,  $\alpha$ ,  $\beta$ , esMax)
2:   if esTerminal(estado) then
3:     return evaluar(estado)
4:   end if
5:   if esMax then
6:      $valor \leftarrow -\infty$ 
7:     for cada movimiento en movimientosValidos(estado) do
8:        $hijo \leftarrow aplicar$ (estado, movimiento)
9:        $valor \leftarrow \text{máx}(valor, MINIMAXAB(hijo, \alpha, \beta, false))$ 
10:       $\alpha \leftarrow \text{máx}(\alpha, valor)$ 
11:      if  $\beta \leq \alpha$  then
12:        break
13:      end if
14:    end for
15:    return valor
16:   else
17:      $valor \leftarrow +\infty$ 
18:     for cada movimiento en movimientosValidos(estado) do
19:        $hijo \leftarrow aplicar$ (estado, movimiento)
20:        $valor \leftarrow \text{mín}(valor, MINIMAXAB(hijo, \alpha, \beta, true))$ 
21:        $\beta \leftarrow \text{mín}(\beta, valor)$ 
22:       if  $\beta \leq \alpha$  then
23:         break
24:       end if
25:     end for
26:     return valor
27:   end if
28: end function

```

3.3. Complejidad

Para el juego de Nim con estado inicial (3, 5, 7):

- Total de palitos: 15
- Factor de ramificación promedio: ≈ 7 (movimientos válidos)
- Profundidad máxima: ≈ 8 (movimientos para terminar)

La memoización reduce significativamente el espacio de búsqueda al evitar reevaluar estados idénticos alcanzados por diferentes caminos.

4. Implementación del Agente

4.1. Arquitectura del Sistema

El sistema se compone de los siguientes módulos:

Cuadro 2: módulos del sistema

módulo	Descripción
<code>nim.py</code>	Logica del juego multi-pila
<code>minimax.py</code>	Algoritmo Minimax con Alpha-Beta
<code>game_agent.py</code>	Agente con niveles de dificultad
<code>graphics.py</code>	Interfaz grafica PyQt6

4.2. Niveles de Dificultad

El agente implementa cuatro niveles de dificultad:

Cuadro 3: Configuración de niveles de dificultad

Nivel	Prob. óptimo	Descripción
Facil	20 %	Movimientos mayormente aleatorios
Medio	50 %	Balance entre óptimo y aleatorio
Dificil	80 %	Casi siempre óptimo
óptimo	100 %	Siempre usa estrategia Nim-sum

En el nivel **óptimo**, el agente utiliza directamente la estrategia de Nim-sum, garantizando un comportamiento determinístico y completamente predecible.

4.3. Proceso de Decisión

1. Obtener el estado actual de las pilas.
2. Según la dificultad, decidir si usar movimiento óptimo.
3. Si es óptimo: calcular Nim-sum y aplicar estrategia matemática.
4. Si no hay movimiento óptimo válido (restricción 1-3): usar Minimax.
5. Si es subóptimo: seleccionar movimiento aleatorio.

5. Resultados Experimentales

5.1. Metodología

Los experimentos se realizaron bajo las siguientes condiciones:

- **Configuración:** Estado inicial (3, 5, 7), máximo 3 palitos por movimiento.
- **Simulaciones:** 100 partidas por nivel de dificultad.
- **Oponente:** Agente aleatorio (selección uniforme de movimientos válidos).
- **Métrica principal:** Tasa de victoria del agente.

5.2. Análisis de Resultados

Los resultados esperados son:

- **Facil ($\sim 20\%$ óptimo):** Tasa de victoria cercana al 60-70 %. Aunque juega mayormente aleatorio, ocasionalmente encuentra movimientos ganadores.
- **Medio ($\sim 50\%$ óptimo):** Tasa de victoria del 80-90 %. El balance permite recuperarse de posiciones subóptimas.
- **Difícil ($\sim 80\%$ óptimo):** Tasa de victoria superior al 90 %. Raramente comete errores.
- **óptimo (100 % óptimo):** Tasa de victoria cercana al 90-95 %. Nota: El jugador humano/aleatorio comienza primero y $(3, 5, 7)$ tiene $\text{Nim-sum} \neq 0$, por lo que el primer jugador tiene ventaja teórica. El agente óptimo gana cuando el oponente comete un error.

5.3. Observación Importante

En el estado inicial $(3, 5, 7)$, el Nim-sum es 1 (posición ganadora). Por lo tanto:

- El **primer jugador** (humano) tiene ventaja si juega óptimamente.
- Si el humano comete un error, el agente óptimo lo capitaliza inmediatamente.
- Un agente óptimo que juega segundo ganará siempre que el oponente no juegue perfectamente.

6. Conclusiones

6.1. Logros del Proyecto

1. Implementación funcional del juego de Nim multi-pila con selección de fila.
2. Interfaz gráfica atractiva con visualización 3D de palitos.
3. Agente inteligente que combina estrategia matemática (Nim-sum) con Minimax.
4. Sistema de dificultad graduada con comportamiento consistente.
5. Framework de Análisis para evaluación del rendimiento.

6.2. Observaciones Clave

1. La estrategia de Nim-sum proporciona la solución óptima sin necesidad de búsqueda.
2. El Minimax sirve como respaldo cuando la restricción de 1-3 palitos impide el movimiento óptimo de Nim-sum .
3. El agente óptimo tiene comportamiento completamente determinístico.
4. La posición inicial favorece al primer jugador matemáticamente.

A. Instrucciones de Uso

A.1. Instalación

```
1 cd nim_minimax_project
2 pip install -r requirements.txt
```

A.2. Ejecución

```
1 cd src
2 python main.py          # Ejecutar juego
```

B. Estructura del Repositorio

```
nim_minimax_project/
  src/
    nim.py           # Logica multi-pila
    minimax.py       # Algoritmo Minimax
    game_agent.py    # Agente inteligente
    graphics.py      # Interfaz PyQt6
    main.py          # Punto de entrada
  docs/
    documentation.tex
    requirements.txt
    README.md
```

Referencias

- [1] Bouton, C. L. (1901). *Nim, A Game with a Complete Mathematical Theory*. Annals of Mathematics, 3(1/4), 35-39.
- [2] Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
- [3] Sprague, R. (1935). *Über mathematische Kampfspiele*. Tohoku Mathematical Journal, 41, 438-444.
- [4] Grundy, P. M. (1939). *Mathematics and Games*. Eureka, 2, 6-8.